

Vector Databases

A1. What is a Vector Database?

A vector database is a database that is mainly used to store and search numerical representations of data called vectors. These vectors are usually created by an embedding model, where text, images, or other data are converted into numbers that represent their meaning.

The main difference from a traditional database like MySQL or PostgreSQL is how the data is searched. In a normal database, we usually search for exact values using things like a B-tree index. For example, we can search for a movie where the title is exactly “Drive”. A vector database, on the other hand, is designed to find data that is similar in meaning. So, if I search for something like “a movie with a great performance”, it can find related movie information even if those exact words are not present.

So, in simple terms, traditional databases are mainly useful for structured and exact lookups, while vector databases are useful when we want to search based on similarity or meaning.

A2. From Raw Text to a Vector Database

For example, consider the sentence:

“Ryan Gosling gave a great performance in the movie Drive.”

The sentence goes through several stages before it can be stored in a vector database.

1. **Raw Text:** First, we have the original sentence as normal human-readable text.
2. **Preprocessing:** The text may be cleaned or prepared before generating its embedding. Depending on the embedding model, this can include operations such as tokenization.
3. **Embedding Generation:** An embedding model converts the sentence into a numerical vector. For example, the sentence might be represented by a vector containing hundreds of numerical values. These numbers capture information about the meaning of the sentence.
4. **Vector Representation:** The generated vector is now the numerical representation of the original sentence. Text with a similar meaning should generally have vectors that are close to each other in the embedding space.
5. **Storage in Vector Database:** Finally, the vector is stored in a vector database along with the original text and usually some metadata, such as the document name or an ID.

So the overall flow is:

Raw Text → Preprocessing → Embedding Model → Vector → Vector Database

A3. Why Can't We Use a Normal B-tree Index for Embeddings?

A normal B-tree index is mainly designed for exact or ordered lookups. For example, if I have a database of movies, I can use an index to quickly find a movie with a particular title or ID.

Embeddings are different because we usually do not want to find an exact vector. We want to find vectors that are *similar* to a query vector. For example, a query about “a romantic movie” should be able to retrieve information about movies that are related to romance, even if the exact words in the query are not present.

This is called a similarity search. Measures such as cosine similarity can be used to compare vectors and find the closest ones.

A B-tree does not naturally perform this type of high-dimensional nearest-neighbor search efficiently. Vector databases use specialized indexing and search techniques designed for finding similar vectors.

So, the main difference is that an exact lookup asks:

“Does this exact value exist?”

while a similarity search asks:

“Which stored vectors are closest to this query vector?”

A4. Cosine Similarity Between Two Vectors

Given:

$$A = [1, 2, 3]$$

$$B = [2, 4, 6]$$

The formula for cosine similarity is:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

First, calculate the dot product:

$$A \cdot B = (1)(2) + (2)(4) + (3)(6)$$

$$= 2 + 8 + 18$$

$$= 28$$

Now calculate the magnitude of A :

$$\|A\| = \sqrt{1^2 + 2^2 + 3^2}$$

$$= \sqrt{1 + 4 + 9} = \sqrt{14}$$

Now calculate the magnitude of B :

$$\begin{aligned}\|B\| &= \sqrt{2^2 + 4^2 + 6^2} \\ &= \sqrt{4 + 16 + 36} = \sqrt{56}\end{aligned}$$

Therefore,

$$\text{Cosine Similarity}(A, B) = \frac{28}{\sqrt{14}\sqrt{56}}$$

Since:

$$\sqrt{14}\sqrt{56} = \sqrt{784} = 28$$

we get:

$$\text{Cosine Similarity}(A, B) = \frac{28}{28} = 1$$

The cosine similarity is therefore:

$$\boxed{1}$$

This means the two vectors point in exactly the same direction. In fact, B is just A multiplied by 2, so they have the same direction even though their magnitudes are different.

A5. Why is a Vector Database Important for RAG?

A vector database is an important part of a RAG system because it allows the system to find relevant information from a large collection of documents.

When a user asks a question, the question is first converted into an embedding. The vector database then compares this query vector with the stored document vectors and retrieves the most similar chunks. These retrieved chunks are then given to the language model as context for generating the answer.

For example, if I have stored information about different Ryan Gosling movies, and I ask:

“Which movie features Ryan Gosling as Ken?”

the query can be converted into a vector and used to retrieve the relevant chunk about *Barbie*. The language model can then use that retrieved information to answer the question.

If we remove the vector database, the retrieval part of RAG is missing. The system would no longer have an efficient way to search through a large collection of embedded documents and find the most relevant information for the query.

So the basic RAG flow is:

Question → Query Embedding → Vector Search → Relevant Chunks → LLM → Answer

Without the vector database or some other retrieval mechanism, RAG loses the part that connects the user’s question with the relevant external knowledge.